

Doc. No.:	P0061R0
Date:	2015-09-16
Reply to:	Clark Nelson
	Richard Smith

Feature-testing preprocessor predicates for C++17

Introduction

SD-6 recommends, in addition to defining feature-test macros, that implementations support two new predicates for use in preprocessor conditional inclusion (like the existing `defined` predicate).

The first of these, `__has_include`, is actually a general-purpose extension to the preprocessor, and would be useful to improve portability between a very wide range of implementations. The other, `__has_cpp_attribute`, has somewhat narrower utility.

SG10 would like EWG and WG21 to consider whether these predicates would be worth adding to the C++17 standard.

Descriptions (verbatim from SD-6)

Testing for the presence of a header: `__has_include`

It is impossible for a C++ program to directly, reliably and portably determine whether or not a library header is available for inclusion. Conditionally including a header requires the use of a configuration macro, whose setting can be determined by a configuration-test process at build time (reliable, but less portable), or by some other means (often not reliable or portable).

To solve this general problem, WG21 recommends that implementers provide, and programmers use, the `__has_include` feature.

Syntax

h-preprocessing-token:

any *preprocessing-token* other than `>`

h-pp-tokens:

h-preprocessing-token

h-pp-tokens *h-preprocessing-token*

has-include-expression:

`__has_include ( header-name )`

`__has_include ( string-literal )`

`__has_include ( < h-pp-tokens > )`

Semantics

In the first form of the *has-include-expression*, the parenthesized *header-name* token is not subject to macro expansion. The second and third forms are considered only if the first form does not match, and the preprocessing tokens are processed just as in normal text.

A *has-include-expression* shall appear only in the controlling constant expression of a `#if` or `#elif` directive ([cpp.cond] 16.1). Prior to the evaluation of such an expression, the source file identified by the parenthesized preprocessing token sequence in each contained *has-include-expression* is searched for as if that preprocessing token sequence were the *pp-tokens* in a `#include` directive, except that no further macro expansion is performed. If such a directive would not satisfy the syntactic requirements of a `#include` directive, the program is ill-formed. The *has-include-expression* is replaced by the *pp-number* 1 if the search for the source file succeeds, and by the *pp-number* 0 if the search fails.

The `#ifdef` and `#ifndef` directives, and the `defined` conditional inclusion operator, shall treat `__has_include` as if it were the name of a defined macro. The identifier `__has_include` shall not appear in any context not mentioned in this section.

Example

This demonstrates a way to use a library optional facility only if it is available.

```
#ifdef __has_include
```

```
# if __has_include(<optional>)
#   include <optional>
#   define have_optional 1
# elif __has_include(<experimental/optional>)
#   include <experimental/optional>
#   define have_optional 1
#   define experimental_optional
# else
#   define have_optional 0
# endif
#endif
```

Testing for the presence of an attribute: `__has_cpp_attribute`

A C++ program cannot directly, reliably, and portably determine whether or not a standard or vendor-specific attribute is available for use. Testing for attribute support generally requires complex macro logic, as illustrated above for language features in general.

To solve this general problem, WG21 recommends that implementers provide, and programmers use, the `__has_cpp_attribute` feature.

Syntax

has-attribute-expression:

```
__has_cpp_attribute ( attribute-token )
```

Semantics

A *has-attribute-expression* shall appear only in the controlling constant expression of a `#if` or `#elif` directive ([cpp.cond] 16.1). The *has-attribute-expression* is replaced by a non-zero pp-number if the implementation supports an attribute with the specified name, and by the pp-number 0 otherwise.

For a standard attribute, the value of the `__has_cpp_attribute` macro is based on the year and month in which the attribute was voted into the working draft. In the case where the attribute is vendor-specific, the value is implementation-defined. However, in most cases it is expected that the availability of an attribute can be detected by any non-zero result.

The `#ifdef` and `#ifndef` directives, and the defined conditional inclusion operator, shall treat `__has_cpp_attribute` as if it were the name of a defined macro. The identifier `__has_cpp_attribute` shall not appear in any context not mentioned in this section.

Example

This demonstrates a way to use the attribute `[[deprecated]]` only if it is available.

```
#ifdef __has_cpp_attribute
# if __has_cpp_attribute(deprecated)
#   define ATTR_DEPRECATED(msg) [[deprecated(msg)]]
# else
#   define ATTR_DEPRECATED(msg)
# endif
#endif
```